

Manuale completo passo dopo passo

BugIt

Agente QA per l'apertura di ticket di bug in VS Code

Configurazione iniziale e uso quotidiano — una guida dettagliata, clic dopo clic, scritta per chi non ha mai fatto nulla del genere. Non serve alcuna competenza tecnica. La segua in ordine e oggi stesso aprirà ticket di bug puliti.

Funziona in: VS Code — Windows (principale), macOS & Linux

COSA TROVERAI

Segua la **Parte 1** una sola volta, in ordine. Tutto il resto può consultarlo quando le serve. Le sezioni **Risoluzione dei problemi** e **FAQ** verso la fine rispondono alle domande più comuni — le controlli prima di scrivere al supporto.

Parte 1 · Configurazione (da fare una sola volta)

Passaggio 0 — Si procuri GitHub Copilot
Passaggio 1 — Installi VS Code
Passaggio 2 — Estragga il download di Buglt
Passaggio 3 — Apra Buglt in VS Code
Passaggio 4 — Selezioni l'agente Buglt in chat
Passaggio 5 — Attivi la sua licenza
Passaggio 6 — Accetti i termini (una volta)

Passaggio 7 — Esegua «Begin setup»

Passaggio 8 — Colleghi il suo tracker

Passaggio 9 — Salvi i suoi accessi

Passaggio 10 — Confermi di essere pronto

Parte 2 · Uso quotidiano

Parte 3 · Lo personalizzi

Parte 4 · Aggiornamenti, backup e sicurezza

Parte 5 · Risoluzione dei problemi e FAQ

Parte 6 · Licenza e assistenza

La regola che la protegge sempre

Buglt **non** apre mai nulla senza prima mostrarle un'anteprima e ottenere il suo «yes» digitato. Ha sempre il controllo. Vuole fare pratica a rischio zero? Digiti `dry run` e scrive l'intero report **senza** aprirlo.

Configurazione iniziale vs. uso quotidiano

Segua tutta la **Parte 1** una sola volta. Dopodiché, aprire Buglt è rapido — e gli aggiornamenti sono un solo comando (`Update`). Veda la Parte 4.

PARTE 1 · CONFIGURAZIONE — DA FARE UNA SOLA VOLTA

Circa 15 minuti. Segua i passaggi in ordine — ognuno si basa sul precedente. Non salti avanti.

0 Si procuri GitHub Copilot

Senza questo non può usare Buglt

Buglt è la "guida" — GitHub Copilot è il "motore" (l'AI che scrive davvero i report). Se non ha Copilot funzionante in VS Code, niente di quanto segue servirà a nulla. Configuri prima questo.

Cos'è: GitHub Copilot è un assistente AI di GitHub (di proprietà di Microsoft). Ha un piano gratuito per un uso occasionale e uno a pagamento (circa \$10/mese) per un uso intenso. Lo installerà al Passaggio 1 e accederà.

1. Le serve un **account GitHub**. Se non ne ha uno, ne crei uno gratuito su github.com/signup.
2. Il piano gratuito di Copilot basta per provare Buglt. Se apre bug tutto il giorno, vale la pena passare al piano a pagamento "Copilot Pro" — può passare in qualsiasi momento su github.com/features/copilot.
3. Attiverà davvero Copilot dentro VS Code al Passaggio 1 — per ora non serve fare altro qui.

1 Installi VS Code

VS Code è un'app gratuita di Microsoft. È la "finestra" in cui vive BugIt.

1. Apri il browser e vada su `code.visualstudio.com`.
2. Clicchi il grande pulsante blu **Download** (rileva automaticamente il suo sistema).
3. Apri il file scaricato e lo installi — **accetti tutte le opzioni predefinite**, basta continuare a cliccare Next/Install.
4. Apri VS Code una volta installato.

Attivi GitHub Copilot dentro VS Code

1. Sulla sinistra di VS Code, clicchi l'icona **Estensioni** (sembra quattro piccoli quadrati).
2. Nella casella di ricerca, digiti **GitHub Copilot**.
3. Clicchi **Install** su "GitHub Copilot" (e, se proposto, anche su "GitHub Copilot Chat").
4. Un messaggio le chiederà di **Sign in to GitHub** — lo clicchi e acceda con il suo account GitHub nella finestra del browser che si apre.

✓ **Saprà che ha funzionato quando:** una piccola icona di Copilot appare in basso in VS Code, e si apre il pannello Chat (l'icona a forma di fumetto a sinistra, oppure preme `Ctrl+Alt+I`).

2 Estragga il download di BugIt

Il suo acquisto include un file .zip (per esempio `bugit-qa-agent.zip`). Deve estrarlo — eseguirlo ancora compresso non funziona.

1. Trovi il file .zip (di solito nella cartella **Download**).
2. **Clic destro** su di esso → scelga **Extract All...** (Windows) oppure faccia doppio clic (Mac).
3. Scelga una posizione semplice come la cartella **Documenti**, poi clicchi Extract.
4. Avrà ora una cartella **BugIt** con tanti file all'interno. Ricordi dove si trova.

Non metterla in OneDrive o in una cartella sincronizzata

Le cartelle sincronizzate nel cloud possono causare conflitti tra i file. La cartella Documenti o il Desktop vanno benissimo. Inoltre non rinomini i file interni — lasci la cartella così com'è.

3 Apri BugIt in VS Code

1. In VS Code, clicchi **File** → **Open Folder** (menu in alto a sinistra).
2. Vada dove ha estratto BugIt (es. Documenti).
3. Clicchi una volta sulla cartella **BugIt** per selezionarla, poi clicchi **Select Folder**.
4. Se VS Code le chiede se si fida degli autori dei file in questa cartella, clicchi **Yes, I trust the authors**.

✓ **Saprà che ha funzionato quando:** vede i file di BugIt (cartelle come `tools`, `.github`, e file come `README.md`) elencati sul lato sinistro di VS Code.

4 Selezioni l'agente Buglt in chat

1. Apra il pannello **Chat** di Copilot — clicchi l'icona a forma di fumetto a sinistra, oppure prema **Ctrl+Alt+I** (Windows) / **Cmd+Alt+I** (Mac).
2. In cima alla casella della chat c'è un piccolo **menu a tendina della modalità** (potrebbe indicare "Ask" o "Agent").
3. Lo clicchi e scelga **Buglt QA Agent** dall'elenco.

Non vede "Buglt QA Agent" nell'elenco?

Si assicuri di aver aperto la cartella Buglt (Passaggio 3), non un'altra cartella — l'agente si carica da lì. Chiuda e riapra la cartella se necessario.

5 Attivi la sua licenza

L'email del suo acquisto contiene una **chiave di licenza** (un codice lungo con trattini e un punto). Questa sblocca Buglt sul suo computer.

1. Nella casella della chat, digiti **hi** e prema Invio.
2. A questo punto Buglt ha bisogno della sua chiave. Per impostazione predefinita, apre una **richiesta mascherata nel terminale** in modo che la chiave non compaia mai a schermo né nel registro della chat — si apre un piccolo pannello del terminale in basso in VS Code; clicchi al suo interno e incolli lì la sua chiave, poi prema Invio (non vedrà nulla di ciò che digita — è normale, è una richiesta in stile password).
3. Preferisce incollarla direttamente nella casella della chat? Va bene anche così — basta dirlo, oppure incollarla come suo primissimo messaggio.

✅ **Saprà che ha funzionato quando:** Buglt risponde "✅ Attivato — licenza a nome di [suo nome/email]."

Tenga privata la sua chiave

La sua chiave è legata a lei. Non la condivida, pubblichi o rivenda — una chiave condivisa può essere disattivata. Una chiave copre solo le sue postazioni. Cambiare computer in seguito non è un problema (veda le FAQ).

6 Accetti i termini (una volta)

La prima volta, Buglt mostra un breve riepilogo e le chiede di accettarlo prima di fare qualsiasi altra cosa.

1. Legga il breve riepilogo che mostra (riveda ogni ticket prima che venga aperto; la sicurezza del suo progetto è una sua responsabilità; le misure di sicurezza sono un aiuto, non una garanzia — e poiché vengono seguite dal modello AI che le risponde in Copilot, un modello più leggero/gratuito potrebbe occasionalmente saltare un passaggio o richiedere che un comando venga ripetuto, cosa che un modello più potente non farebbe).
2. Risponda **I agree** e prema Invio.

✅ **Saprà che ha funzionato quando:** Buglt conferma e passa alla configurazione. Glielo chiederà solo questa volta — se lo ricorda.

7 Esegua «Begin setup»

Ora Buglt si configura da solo intervistandola in linguaggio semplice. Non modifica mai un file di impostazioni a mano.

Farà brevi domande e attiverà solo ciò che sceglie. La più importante riguarda il suo tracker:

Le chiede...	Cosa rispondere
Quale tracker usa? (ne scelga uno — obbligatorio)	Dove il suo team tiene i bug. Jira e Azure DevOps ricevono una configurazione guidata con mappatura dei campi testata — gravità/priorità viene impostata automaticamente in base al suo report. Jira usa l'Atlassian Roov MCP con accesso da browser; Azure DevOps usa l'MCP remoto di Microsoft con ambito sull'organizzazione (un'anteprima pubblica), anch'esso con accesso da browser. Sentry, GitHub, Linear e Notion usano un endpoint noto che BugIt configura e verifica in tempo reale sul suo account — li consideri sperimentali finché quei controlli non vengono superati. Tutti gli altri (GitLab, Shortcut, YouTrack, Bugzilla, ClickUp, Asana, Trello) si collegano tramite il suo server MCP compatibile, che BugIt la aiuta a configurare. Scelga quello che usa già.
Strumento per i crash? (facoltativo)	Sentry usa un endpoint noto che BugIt verifica in tempo reale sul suo account (sperimentale); Crashlytics, BugSnag, Raygun, Rollbar, App Center e Datadog si collegano tramite il suo server MCP compatibile — solo se il suo team ne usa uno.
Gestione dei test? (facoltativo)	TestRail, Xray, Zephyr, Qase.
Pubblicare i bug in chat? (facoltativo)	Slack, Teams, Discord, o email.
Specifiche / conoscenza? (facoltativo)	Confluence, Notion, SharePoint, Google Docs, o una cartella locale di file Markdown in questo repository.
Lingue	La sua lingua principale (inglese di default). Le chiede con un semplice sì/no se apre ticket anche in una seconda lingua — la maggior parte dei team ne usa solo una.
Funzionalità avanzate? (facoltativo)	Strumenti extra come il digest di triage, l'annullamento e l'esportazione offline — veda la Parte 2. Tutti facoltativi.

Suggerimento: parta da un template

Se il suo lavoro rientra in una categoria, digiti `preset.py game` (anche `web-saas`, `mobile`, `enterprise`) per precompilare categorie e impostazioni sensate. Potrà sempre cambiarle in seguito.

La configurazione si interrompe? Ripetila e basta

Se `Begin setup` si interrompe a metà — ha chiuso VS Code, oppure si è solo messa in pausa — digitare di nuovo `Begin setup` riprende esattamente da dove si era fermata. Non le farà rispondere di nuovo a domande a cui ha già risposto. Per cambiare deliberatamente qualcosa in seguito (aggiungere un tracker, cambiare una scelta), lo dica semplicemente — per esempio "aggiungi un tracker" — e riapre quella parte della configurazione.

8 Colleghi il suo tracker (il "ponte")

Il suo tracker comunica con BugIt tramite un piccolo ponte (il suo nome tecnico è "server MCP"). BugIt lo configura **per lei** — lei risponde solo a delle domande e clicca un pulsante. Non modifica mai i file.

8a · Lasci che BugIt configuri il ponte

- 1. Durante la configurazione, BugIt propone di collegare il suo tracker. Per Jira e Confluence (Atlassian RoVo) conosce la connessione guidata; per Sentry, GitHub, Linear e Notion conosce l'endpoint standard e lo verifica in tempo reale sul suo account — **confermi** quando glielo chiede, ed esegue i controlli prima che lei ci faccia affidamento (li consideri sperimentali finché non vengono superati).
- 2. Per gli strumenti self-hosted o forniti dall'organizzazione, le dice in una frase dove trovare l'indirizzo del suo server MCP compatibile e le chiede di **incollarlo in chat** — poi lo scrive nelle impostazioni.

8b · Attivi il ponte

- 1. Nell'elenco dei file a sinistra, apra il file `.vscode/mcp.json` (clicchi una volta).
- 2. Al suo interno, cerchi il piccolo testo grigio `▶ Start | More...` — si trova appena sopra il nome di ogni server. È piccolo e facile da non notare.
- 3. Clicchi **Start** su ogni server che BugIt le ha detto di avviare (di solito solo il suo tracker).

8c · Acceda quando richiesto

La prima volta che un ponte si collega, deve dimostrare che è davvero lei. Succede una di queste due cose:

- **Si apre una finestra del browser** → acceda al suo tracker normalmente (comune per GitHub e Azure DevOps).
- **Compare una piccola finestra in alto in VS Code** → chiede la sua email e/o un token di accesso. Lo incolli e preme Invio.

Dove trovo un token di accesso?

Un token è come una password monouso fornita dal suo tracker. Ne crei uno sul sito del suo tracker, poi lo incolli quando la finestra lo richiede. Ecco le pagine per i più diffusi — acceda con l'account che usa normalmente:

Il suo tracker	Dove creare un token
Jira / Confluence (Atlassian)	<code>id.atlassian.com/manage-profile/security/api-tokens</code>
GitHub	<code>github.com/settings/tokens</code> (oppure accedi direttamente dal browser)
GitLab	<code>gitlab.com/-/user_settings/personal_access_tokens</code>
Sentry	sentry.io → Settings → Account → API → Auth Tokens
Linear	linear.app → Settings → API → Personal API keys
Azure DevOps	Di solito accede tramite browser — nessun token necessario.

Copi il suo token appena viene mostrato

La maggior parte dei servizi mostra un nuovo token una sola volta. Lo copi subito e lo conservi in un posto sicuro finché BugIt non lo salva per lei (Passaggio 9). Scelga la scadenza più lunga che il suo tracker offre, così non dovrà rifarlo a breve.

8d · Verifichi che sia collegato

Dopo aver avviato un ponte, il suo testo grigio cambia in "Running" oppure mostra un punto verde. Per esserne sicuro, digiti in chat:

LEI DIGITA

Check status

✅ **Saprà che ha funzionato quando:** Buglt elenca il suo tracker come collegato / in buono stato.

Il ponte si disattiva quando chiude VS Code

Ogni volta che riapre VS Code, clicchi di nuovo **Start** sul suo server in `.vscode/mcp.json`. I suoi accessi salvati vengono ricordati (Passaggio 9) — deve solo riavviare il ponte. È un comportamento di VS Code, non un problema di Buglt.

9 Salvi i suoi accessi

Così non dovrà mai più cercare il suo token, lasci che Buglt lo memorizzi in modo sicuro nel vault integrato del suo computer.

LEI DIGITA

save my tokens

Per impostazione predefinita, Buglt apre ancora una volta una **richiesta mascherata nel terminale** per la sua email/token, così non tocca mai il registro della chat — lo incolli lì quando si apre (può anche chiedergli di riceverlo direttamente in chat, se preferisce). In entrambi i casi, li salva nel vault delle credenziali del suo sistema operativo (Windows Credential Manager / macOS Keychain / Linux Secret Service) — mai in un file in chiaro, mai inviati da nessuna parte. La prossima volta che un ponte glielo chiede, digiti semplicemente:

LEI DIGITA

show my tokens

...e Buglt mostra i suoi dati salvati direttamente nel suo editor (solo per lei, mai inviati da nessuna parte) da copiare e incollare. Niente più ricerche sul sito del suo tracker.

10 Confermi di essere pronto

Chieda a Buglt di ricontrollare tutto prima di aprire ticket sul serio:

LEI DIGITA

Check readiness

Elencare tutto ciò che manca ancora (di solito solo "avvia il suo ponte" o "accedi"). Quando è tutto verde, vedrà "✅ Configurazione completata."

Configurazione fatta — per sempre

Non ripeterà la Parte 1. Da qui in poi, aprire BugIt significa: apra la cartella → avvii il suo ponte in `.vscode/mcp.json` → digiti `hi`. Per cambiare una scelta in seguito, digiti semplicemente `Begin setup` e dica cosa cambiare.

Niente GitHub Copilot? C'è una procedura guidata da terminale

Se non può usare Copilot, apra il Terminale integrato (Terminal → New Terminal) ed esegua `python tools/setup_wizard.py`. Fa qualche domanda e scrive le sue impostazioni senza l'AI. Può anche eseguire BugIt in modalità standalone con la sua chiave AI — veda le FAQ.

PARTE 2 · USO QUOTIDIANO

Una volta configurato, lavora interamente dalla chat. Parli in modo naturale, oppure usi gli esempi esatti qui sotto. Digiti `help` in qualsiasi momento per l'elenco completo.

Scriva un report di bug

Il compito principale di BugIt. Descriva il bug in una frase semplice; scrive il ticket completo, compila automaticamente versione e categoria, controlla i duplicati, le mostra un'anteprima e lo apre dopo il suo `yes`.

LEI DIGITA

Write a bug report: l'app si chiude ogni volta che tocco Salva su iPhone

Prepara un titolo, i passaggi per riprodurlo, il risultato atteso e quello effettivo, la gravità e la piattaforma — e imposta sempre il campo di priorità/gravità del tracker stesso in base a questi dati, non un valore predefinito generico. Vuole delle modifiche prima di aprirlo? Lo dica e basta — per esempio "aumenta la gravità" oppure "aggiungi che è iniziato con l'ultima build."

Bug rapidi (apertura veloce)

Per le casistiche più comuni (crash, layout, lentezza, elemento mancante, blocco...), il modulo rapido salta alcune domande e compila categoria e priorità per lei — e comunque esegue prima il controllo completo dei duplicati.

LEI DIGITA (ESEMPI)

Quick bug: crash all'avvio dopo l'aggiornamento

Quick bug: layout il pulsante si sovrappone al testo nella schermata delle impostazioni

Quick bug: blocker non riesco a proseguire oltre la fase di pagamento

Quick bug: slow la dashboard impiega 20 secondi a caricarsi

Report di bug per i crash

Se ha collegato uno strumento per i crash (come Sentry), BugIt fa tutto quanto sopra e in più abbina il suo report al crash e recupera lo stack trace per lei.

LEI DIGITA

Write a crash bug report: il gioco va in crash quando apro la mappa

Commenti di verifica e chiusura

Dopo aver verificato una correzione, BugIt scrive un commento ordinato da pubblicare sul ticket. Scrive (ed eventualmente pubblica) il commento — non cambia lo stato del ticket. È lei a chiudere/risolvere/riaprire il ticket nel suo tracker.

Lei digita	Cosa ottiene
<code>Close #123</code>	Chiede quale scenario (correzione confermata, chiuso su richiesta, comportamento previsto, riapertura...) poi scrive quel commento.
<code>Write a verification comment for 1.2.0 on Windows</code>	Un commento "correzione confermata" con i dettagli della build già compilati.
<code>Write a reopen comment for 1.2.0 on Windows</code>	Un commento "il problema persiste" (poi riapre lei stesso il ticket).

Traduzione

Traduce report o termini tra le lingue configurate, usando la formulazione esatta del suo team (dal suo glossario — veda la Parte 3).

LEI DIGITA (ESEMPI)

```
Translate to ja: il pulsante salva non fa nulla nella schermata del profilo
Translate this bug report to English: [incolla il testo]
```

Cerchi informazioni (specifiche e glossario)

Se ha collegato le specifiche o compilato il suo glossario, faccia domande a BugIt sul suo progetto.

LEI DIGITA (ESEMPI)

```
Cos'è [il suo termine]?
Guidami attraverso il flusso di checkout
Cosa c'è di nuovo nelle specifiche?
```

Rilevamento dei duplicati

Avviene automaticamente prima di ogni apertura — BugIt cerca nel suo tracker ticket corrispondenti (anche formulati in modo un po' diverso) e la avvisa, così non apre mai due volte lo stesso bug.

Funzionalità avanzate (se le ha attivate)

Digiti questo	Cosa fa
<code>Triage digest</code>	Standup istantaneo: bug aperti per area/gravità, i più vecchi segnalati.
<code>Export bug to file</code>	Salva il report come Markdown/CSV/JSON invece di aprirlo (ottimo prima che il suo tracker sia configurato).
<code>Undo last filing</code>	Annulla l'ultimo ticket/commento, dove il suo tracker lo consente.
(automatico)	Segnalazioni SLA sui bug urgenti · campi specifici per categoria.

L'elenco completo dei comandi

LEI DIGITA

help

...mostra ogni comando disponibile nell'agente, in qualsiasi momento.

Faccia pratica in sicurezza in qualsiasi momento

Digiti `dry run` e BugIt scrive l'intero report **senza** aprire nulla — perfetto per imparare o fare test.

PARTE 3 · LO PERSONALIZZI — AGGIUNGA LA CONOSCENZA DEL SUO PROGETTO

Appena installato, BugIt è un agente QA a foglio bianco. Ecco come insegnargli il suo progetto dopo l'acquisto. Nessuna programmazione — quasi solo parlargli in chat. Tutto ciò che aggiunge resta sul suo computer.

1 · Le parole del suo team (il glossario)

Questo rende accurate le traduzioni e l'individuazione della categoria per il suo progetto.

1. Nell'elenco dei file, apra `.github/glossary/terms.template.md`.
2. Compili la tabella con le parole del suo team — nomi delle funzionalità, nomi delle schermate, nomi degli elementi, abbreviazioni e (se usa due lingue) le loro traduzioni.
3. Salvi il file (`Ctrl+S`). Tutto qui — da questo momento BugIt usa esattamente questi termini.

Scorciatoia

Attivi l'"auto-popolamento del glossario" durante `Begin setup` e BugIt suggerisce termini dai suoi ticket esistenti — lei approva ognuno.

2 · Il suo stile dei ticket (house style)

Vuole che i ticket rispecchino esattamente il formato del suo team — stile del titolo, etichette di gravità, campi obbligatori? Non lo descrive — lo **mostra**:

1. In chat, dica: "adattati allo stile del mio team".
2. Incolli **uno screenshot** di un ticket esistente dal suo tracker.
3. BugIt lo analizza (in locale, con eventuali dati personali rimossi prima) e salva il suo formato, poi lo segue in ogni ticket futuro.

3 · Le sue categorie, piattaforme e campi

Lo dica semplicemente a BugIt in linguaggio naturale e aggiorna le sue impostazioni per lei:

LEI DIGITA (ESEMPI)

Le mie categorie sono Gameplay, UI, Audio e Networking
Facciamo test su Windows e Xbox

4 • I suoi strumenti personalizzati (connessioni personalizzate)

Usa uno strumento che non è nell'elenco integrato? Lo colleghi proprio come uno predefinito:

LEI DIGITA

Add a custom MCP server

Gli dia un nome breve e il suo indirizzo (deve iniziare con `https://`); BugIt lo collega e ne verifica lo stato di salute per lei.

5 • Lo corregga semplicemente man mano

Il modo più semplice di tutti: quando BugIt prepara un ticket, corregga qualsiasi cosa non le piaccia — un'etichetta, la formulazione, la gravità. Con l'opzione "impara dalle tue modifiche" attivata (consigliata), ricorda la sua preferenza e la applica la volta successiva. La conoscenza del suo progetto si accumula naturalmente — e non lascia mai la sua macchina.

Tutto ciò che aggiunge è suo e resta in locale

Il suo glossario, lo stile dei ticket, le categorie e le preferenze apprese vivono sul suo computer, vengono salvati in backup prima di ogni aggiornamento e non vengono mai inviati a nessuno.

6 • Condividi la sua configurazione con il team (licenza Team)

Una volta impostati glossario, stile dei ticket e scelta del tracker come preferisce, dia al resto del suo team esattamente la stessa configurazione con un solo comando, invece di ripetere i passaggi 1–3 su ogni macchina:

LEI ESEGUE (UNA VOLTA, DOPO CHE LA SUA CONFIGURAZIONE È COME LA VUOLE)

```
python tools/share.py export
```

Questo scrive `config.share.zip` — le sue scelte di tracker/crash/comunicazioni, il glossario e lo stile dei ticket, raggruppati insieme senza password o token all'interno. Lo invii al suo team.

OGNI COLLEGA ESEGUE

```
python tools/share.py import config.share.zip
```

Otengono immediatamente la sua terminologia esatta, il suo stile dei ticket e la configurazione del tracker — senza ridigitare le categorie, senza reincollare uno screenshot per lo stile. La loro licenza e le impostazioni del dispositivo restano invariate.

Se l'importazione cambia dove vengono inviati i dati

Se una configurazione condivisa aggiunge una nuova connessione personalizzata o un indirizzo di notifica, BugIt mostra esattamente cosa è cambiato prima che accada qualsiasi altra cosa — non viene applicato silenziosamente. Inoltre crea prima un'istantanea della configurazione attuale del collega, così `Restore my settings` annulla l'importazione se non è quello che si aspettava.

PARTE 4 · AGGIORNAMENTI, BACKUP E SICUREZZA

Ricevere gli aggiornamenti (un solo comando)

Quando è disponibile una nuova versione, BugIt glielo segnala una volta quando digita `hi`. Gli aggiornamenti gratuiti della v1.x sono inclusi finché la sua licenza è attiva. Per installarla:

LEI DIGITA

Update

1. BugIt esegue prima un backup aggiornato dei suoi file.
2. Scarica la nuova versione e verifica che sia autentica e non manomessa (è firmata crittograficamente — un aggiornamento falso o alterato viene rifiutato).
3. La installa senza toccare le sue impostazioni, il glossario, lo stile dei ticket, la licenza o i token.
4. Le chiede di ricaricare VS Code (`Ctrl+Shift+P` → digiti "Reload Window" → Invio). Avvia una nuova chat ed è sulla nuova versione.

Mai eliminare cartelle

A differenza della prima installazione, gli aggiornamenti avvengono sul posto. Non elimina né ricarica mai nulla, e la sua configurazione viene sempre preservata e salvata in backup.

Cosa è sicuro modificare a mano — e cosa no

Sicuro, e conservato a ogni aggiornamento: `config.json`, `.github/instructions/house-style.instructions.md` (veda la Parte 3 — questo è il modo supportato per personalizzare il comportamento), `.github/glossary/terms.template.md`, `.github/instructions/learned.instructions.md` e `.vscode/mcp.json`.

Non sicuro — non li modifichi a mano

Il file dell'agente (`.github/agents/bugit-qa-agent.agent.md`), qualsiasi altro file sotto `.github/instructions/` e tutto ciò che si trova in `tools/` sono il prodotto stesso, non le sue impostazioni. Un aggiornamento li **sovrascrive silenziosamente**, e — a differenza dei file sopra — **non esiste un backup da cui ripristinarli**. BugIt controlla questo all'avvio e di nuovo subito prima di installare un aggiornamento, e la avvisa se trova una modifica — ma la regola più sicura è semplicemente non modificarli.

Backup e ripristino

Lei digita	Cosa fa
<code>Back up my settings</code>	Salva un'istantanea locale della sua configurazione, del glossario, dello stile dei ticket, della licenza e delle connessioni.
<code>List backups</code>	Mostra ogni istantanea salvata con la sua data.
<code>Restore my settings</code>	Ripristina un'istantanea (e salva prima un'istantanea del suo stato attuale, così anche un ripristino è annullabile).

Automatico prima di ogni aggiornamento

BugIt esegue sempre un backup prima di installare una nuova versione, così un aggiornamento non può mai far perdere la sua configurazione. Se qualcosa non sembra giusto, `Restore my settings` lo sistema.

Sicurezza — cosa è protetto

✓ Nulla senza il suo yes

Ogni ticket e commento viene mostrato in anteprima; qualsiasi cosa irreversibile richiede una conferma digitata.

🔧 Dry-run = zero scritture

Dica `dry run` e BugIt si limita a leggere — non apre mai ticket, non commenta, non invia notifiche.

🔑 Nessun segreto nei file

I token vivono nel vault del suo sistema operativo, mai in un file in chiaro, mai inviati a noi.

🔒 Funziona sulla sua macchina

Comunica solo con gli strumenti che collega e il suo modello AI.

I suoi dati e la privacy

L'unica cosa che BugIt invia a Taskivator sono i dati di licenza/aggiornamento: la sua chiave di licenza, un ID dispositivo con hash a senso unico (un codice cifrato che non può identificarla) e — solo se ne ha impostata una in configurazione — l'etichetta di postazione che ha scelto (ad es. un nome o un'email, così può distinguere i suoi dispositivi; non deve mai essere reale). I suoi report di bug, le specifiche, il glossario, le impostazioni e i token non lasciano mai la sua macchina. Nessuna telemetria, nessuna analisi, mai. I dettagli completi sono nel file `PRIVACY.md` nella sua cartella BugIt.

Rischi e avvertenze importanti — legga con attenzione

L'AI può sbagliare

BugIt scrive i report con un modello AI, che può interpretare male i dettagli o inventare qualcosa. Legga sempre l'anteprima prima di dire "yes". È lei ad approvare ciò che viene aperto.

Apre ticket sul suo tracker reale

Una volta approvato, viene creato un ticket vero. Sta facendo pratica? Usi `dry run` per scrivere il report senza aprirlo.

La sicurezza del suo progetto è sua

Come usa BugIt, e la sicurezza dei suoi progetti, dati e tracker, sono una sua responsabilità. Le misure di sicurezza riducono il rischio ma non sostituiscono la sua revisione.

Cosa non fa BugIt

La mappatura dei campi testata (gravità/priorità impostate automaticamente) riguarda Jira + Azure DevOps (Azure DevOps tramite l'anteprima pubblica dell'MCP remoto di Microsoft); Confluence usa la stessa connessione Atlassian guidata; Sentry, GitHub, Linear e Notion sono sperimentali e verificati in tempo reale sul suo account; tutto il resto si collega tramite il suo server MCP compatibile (che configura lei, con l'aiuto di BugIt); usa la sua AI (Copilot o la sua chiave OpenAI/Anthropic), non un modello incluso; l'oscuramento dei dati è best-effort; e funziona solo in VS Code. Una licenza = un dispositivo (Solo) o cinque (Team). I risultati e la coerenza dipendono anche da quale modello AI risponde dentro Copilot — un modello più potente segue i passaggi di sicurezza in modo più affidabile di uno molto leggero/gratuito.

PARTE 5 · RISOLUZIONE DEI PROBLEMI E FAQ

Quasi ogni intoppo si risolve in pochi secondi — e la soluzione più rapida è di solito chiederlo direttamente all'assistente.

🌟 Provi prima questo — chiedi all'AI di risolverlo

Ogni volta che qualcosa va storto — durante la configurazione o nell'uso quotidiano — la soluzione più rapida è quasi sempre **chiedere all'assistente direttamente in chat**. Descriva cosa è successo con parole semplici (per esempio, "la configurazione si è fermata a metà" oppure "il ponte del mio tracker non si connette") e gli chiedi di risolverlo. L'AI può diagnosticare e riparare la maggior parte dei problemi sul momento. Provi questa strada prima di scriverci un'email — risolve la grande maggioranza dei problemi in pochi secondi.

Applicare una correzione dell'AI: il pulsante "Keep"

Se l'assistente risolve qualcosa modificando un file, VS Code mostra una piccola barra Keep / Undo. Se ha chiesto lei la correzione e ne è soddisfatto, clicchi **Keep** — la modifica viene applicata solo alla sua copia, sul suo computer. Clicchi **Undo** per scartarla. Nulla che tiene con Keep viene condiviso con nessun altro.

Cosa vede	Cosa fare
"Attiva per iniziare"	Per impostazione predefinita BugIt apre una richiesta mascherata nel terminale per la sua chiave; la incolli lì (oppure la incolli in chat, se preferisce). Non ha una chiave? Ne acquisti una sullo store.
"Nessun tracker abilitato"	Digiti <code>Begin setup</code> e scelga il suo tracker.
Il suo ponte mostra una X rossa / non si connette	Apra <code>.vscode/mcp.json</code> e clicchi Start sopra il server. Ancora bloccato? Digiti <code>fix servers</code> e segua i passaggi.
Continua a chiederle di accedere	Digiti <code>show my tokens</code> — mostrato in locale, solo per lei da copiare — e incolli il valore nella finestra. (Prima volta? Veda il Passaggio 8c.)
Non apre un bug	Digiti <code>Check readiness</code> — elenca esattamente cosa manca (di solito il ponte non è avviato o non ha eseguito l'accesso).
Qualcosa sembra rotto dopo un aggiornamento	Digiti <code>Restore my settings</code> per tornare indietro.
L'agente sembra "fuori copione"	Digiti <code>Read and follow all your given instructions</code> , oppure avvii una nuova chat. Questo può succedere più spesso con un modello AI leggero/gratuito — un modello più potente nel selettore di Copilot è più coerente.
Le risposte sembrano generiche / non specifiche per il progetto	Gli dia più contesto, oppure gli mostri un ticket esistente durante <code>Begin setup</code> così impara il suo stile. Riesegua <code>Begin setup</code> per affinarlo.
Vuole cambiare una scelta di configurazione	Digiti <code>Begin setup</code> e dica cosa cambiare (per esempio "aggiungi un tracker" o "ricomincia da capo") — riapre solo quella parte. Se la configurazione è stata semplicemente interrotta a metà, un semplice <code>Begin setup</code> riprende da dove si era fermata invece di ricominciare.

Controlli di salute integrati

`Check status` (i miei strumenti sono collegati?) · `Check readiness` (cosa manca prima di poter aprire ticket?).
Per un controllo più approfondito, apra Terminal → New Terminal ed esegua `python tools/selftest.py`.

Domande frequenti

Devo sapere programmare?

No. Se sa installare un'app e scrivere una frase, può usare BugIt. Si occupa lui delle parti tecniche.

Aprirà bug senza che io lo sappia?

Mai. Ogni ticket e commento viene mostrato in anteprima e richiede il suo "yes" digitato. Usi `dry run` per fare pratica senza alcuna scrittura.

Devo avviare il ponte ogni volta?

Sì — apra `.vscode/mcp.json` e clicchi Start quando riapre VS Code. I suoi accessi salvati vengono ricordati; solo il ponte deve essere riavviato.

Posso usarlo su due computer?

Uno alla volta con Solo (cinque con Team). Cambiare dispositivo va bene — una licenza può passare da un dispositivo all'altro una volta ogni 24 ore. Digiti `License status` per verificare.

Cosa succede se il server delle licenze è offline?

Continua a lavorare. BugIt funziona con un accesso memorizzato in cache per un massimo di 7 giorni offline, così un'interruzione — o semplicemente un weekend senza internet — non la blocca mai a metà turno.

Mi serve GitHub Copilot?

È il modo più semplice. Se non lo ha, esegua `python tools/setup_wizard.py` nel Terminale per configurarlo, e BugIt può funzionare in modalità standalone con la sua chiave OpenAI/Anthropic (`python standalone/run.py`).

I miei dati sono privati?

Sì. L'unica cosa che BugIt invia a Taskivator sono i dati di licenza/aggiornamento — la sua chiave, un ID dispositivo con hash e (solo se ne ha impostata una) l'etichetta di postazione scelta — nessuna telemetria, mai. I suoi ticket vanno solo al modello AI e al tracker che collega, mai a noi.

Posso modificare i file di BugIt?

È lei a dover personalizzare le sue parti — il glossario, lo stile dei ticket e le impostazioni (Parte 3). Solo, non disattivi la licenza/attivazione. Se trova un vero bug nello strumento stesso, scriva al supporto così può essere corretto per tutti nel prossimo aggiornamento.

Con quali tracker funziona?

Jira e Azure DevOps ricevono una configurazione guidata con mappatura dei campi testata (gravità/priorità impostate automaticamente) — Jira tramite l'Atlassian RoVo MCP, Azure DevOps tramite l'anteprima pubblica dell'MCP remoto di Microsoft. Sentry, GitHub, Linear e Notion usano un endpoint noto che BugIt configura e verifica in tempo reale sul suo account (sperimentale finché quei controlli non vengono superati); GitLab, Shortcut, YouTrack, Bugzilla, ClickUp, Asana e Trello si collegano tramite il suo server MCP compatibile — BugIt scrive il ticket e lo apre tramite qualsiasi connettore lei attivi. Scegli il suo durante la configurazione e lo cambi in qualsiasi momento con `Begin setup` .

Rileverà i bug duplicati prima che io apra un ticket?

Sì. Prima che venga scritto qualsiasi cosa, cerca nel suo tracker report simili — anche formulati in modo un po' diverso — e le mostra le corrispondenze, così può evitare di segnalare due volte lo stesso bug. Decide comunque lei se procedere.

Può scrivere ticket in più di una lingua?

Sì. Aggiunga una seconda lingua durante la configurazione e ogni report viene scritto in entrambe, mantenute in blocchi separati, usando il suo glossario così i termini di prodotto restano esatti — non improvvisa mai una traduzione.

Posso allegare uno screenshot?

Basta incollare l'immagine in chat. Buglt la legge, rimuove qualsiasi dato sensibile che nota e la allega al ticket una volta che approva.

Nasconde password e token nei miei report?

Quando l'oscuramento è attivo, ripulisce email, token e indirizzi IP da ogni bozza prima di aprirla. Vede sempre il testo completo nella sua anteprima per primo.

Ho aperto un ticket per errore — posso annullarlo?

L'anteprima e il "yes" digitato bloccano la maggior parte degli errori prima che venga scritto qualsiasi cosa. Se ha attivato il log di controllo, digiti `Undo last filing`. Altrimenti, chiuda o modifichi il ticket nel suo tracker come al solito.

Può aiutarmi a verificare o chiudere un bug?

Sì. Chieda un commento di verifica e prepara una nota chiara di superato/non superato (nelle sue lingue) sul ticket esistente — la approva lei prima che venga pubblicata.

Posso usarlo per più di un progetto?

Ogni cartella Buglt è configurata per il tracker di un solo progetto. Per un altro progetto, riesegua `Begin setup` per puntarlo altrove, oppure mantenga una copia separata per ogni progetto.

Come ricevo gli aggiornamenti?

Quando esce una nuova versione, Buglt glielo segnala non appena avvia una chat. Digiti `update` e la installa sul posto — il suo glossario, lo stile dei ticket e le impostazioni vengono conservati, e li salva prima in backup.

Cosa succede quando la mia licenza scade?

Non viene tagliato fuori subito. Buglt continua a funzionare per un periodo di tolleranza di 14 giorni e mostra quanti giorni restano (digiti `License status` in qualsiasi momento). Rinnovi entro quei 14 giorni e non subirà mai interruzioni; una volta terminato il periodo di tolleranza, attivi una chiave rinnovata per continuare ad aprire ticket. Questo è separato dall'accesso offline di 7 giorni descritto sopra, che copre solo un'interruzione del *server* delle licenze.

Se rinnovo o acquisto un'altra licenza, i giorni si sommano?

No — le licenze non si sommano. Attivare una nuova chiave **sostituisce** quella attuale; il suo periodo inizia da zero dal giorno dell'attivazione, e gli eventuali giorni non utilizzati della vecchia chiave non vengono riportati. Attivi quindi un rinnovo verso la fine del suo periodo attuale, non in anticipo.

PARTE 6 · LICENZA E ASSISTENZA

Termini di licenza (riepilogo in linguaggio semplice)

I termini completi e vincolanti si trovano nel file `LICENSE` nella sua cartella Buglt — quel documento fa fede. In breve:

Argomento

In parole semplici

Concesso in licenza, non venduto	Acquista una licenza d'uso di BugIt, non la proprietà. Taskivator mantiene tutti i diritti.
Le sue postazioni	Solo = un dispositivo alla volta; Team = cinque. La sua chiave è personale — non la condivide, rivenda o pubblici.
Revoca	Una chiave condivisa, rimborsata, contestata o usata impropriamente può essere revocata.
Personalizza, ma...	Modifichi liberamente configurazione, glossario e template — ma non disattivi né aggiri la licenza/attivazione.
La sua responsabilità	Come usa BugIt e la sicurezza del suo progetto sono sue. Riveda e approvi ogni report prima che venga aperto. Le misure di sicurezza sono un aiuto, non una garanzia.
"Così com'è", responsabilità	Fornito così com'è, senza garanzia. Nella misura consentita dalla legge, la responsabilità è limitata a quanto pagato, escludendo danni indiretti/conseguenziali.
Durata e legge	Una licenza annuale che inizia il giorno in cui la attiva, regolata dalle leggi del Giappone. I rimborsi vengono gestiti tramite lo store presso cui ha acquistato.
I rinnovi non si sommano	Attivare una nuova chiave sostituisce quella attuale — il suo periodo inizia da zero all'attivazione e i giorni non utilizzati non vengono riportati, quindi rinnovi verso la fine del suo periodo. Quando un periodo termina, un periodo di tolleranza di 14 giorni le permette di continuare a lavorare mentre rinnova, prima che l'apertura dei ticket si blocchi.

Due documenti nella sua cartella

[LICENSE](#) (termini completi) e [PRIVACY.md](#) (informativa sulla privacy). Attivando e utilizzando BugIt, accetta la LICENSE.

Ottenere assistenza

Controlli prima la Risoluzione dei problemi e le FAQ sopra — coprono quasi tutto. Poi, in quest'ordine:

1 • Chieda all'AI di risolverlo ★

La sua prima mossa migliore: descriva il problema in chat e chieda all'assistente di risolverlo. Se modifica un file e le sembra corretto, clicchi Keep per applicarlo (solo sul suo PC).

2 • Esegua un controllo di salute

[Check status](#) · [Check readiness](#) · oppure [python tools/selftest.py](#) nel Terminale.

3 • Ripristini

[Restore my settings](#) annulla una modifica o un aggiornamento andato male.

4 • Contatti una persona

Solo se la guida e l'AI non hanno potuto aiutarla: visiti [bugit.dev](#) o scriva a support@bugit.dev (l'assistenza è gestita in inglese). Problemi di configurazione nei primi 7 giorni? La aiuteremo a partire o con un rimborso tramite lo store.

Tenga le informazioni riservate del progetto fuori dalle email di supporto

Ogni progetto è diverso, e gran parte del suo lavoro potrebbe essere riservato. Se ci scrive, descriva il problema di BugIt solo in termini generali — non incolli codice riservato, dettagli sui bug, specifiche, screenshot o dati del suo progetto. Taskivator non è responsabile per eventuali informazioni riservate che ci vengono inviate, e qualsiasi contenuto riservato che riceviamo viene eliminato immediatamente. Quando possibile, chieda prima aiuto all'assistente AI — può risolvere la maggior parte dei problemi direttamente nel suo editor.

Grazie per aver scelto BugIt.

Apra ticket puliti, salti i duplicati, e si riprenda il suo tempo — un bug alla volta.